

# SPB - Student Placement Bridge

## Technical Architecture Review Document

---

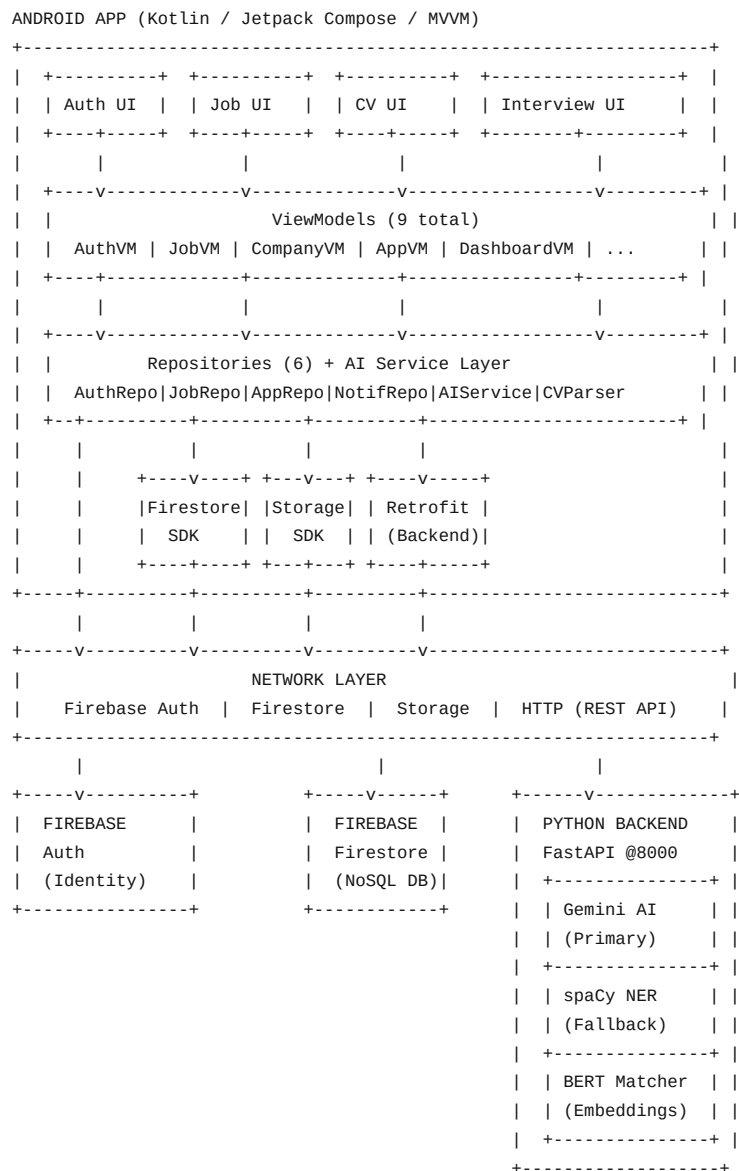
Version 1.0.0 | June 2026

SPB Project - Final Year Capstone

*This document provides a comprehensive technical architecture review of the Student Placement Bridge (SPB) system, covering the Android application architecture, Python backend services, database schema, AI pipeline, security model, and deployment strategy.*

## SECTION 2: System Architecture Overview

The SPB system follows a client-server architecture with three primary tiers: an Android frontend built with Kotlin and Jetpack Compose following MVVM patterns, a Firebase backend providing authentication and real-time database services, and a Python FastAPI backend that hosts the AI/ML pipeline including CV parsing, semantic matching, interview scoring, and job recommendations.



The architecture separates concerns into clear layers: Presentation (UI/Compose screens), Application (ViewModels with StateFlow), Data (Repositories accessing Firebase/FastAPI), and AI (Gemini/spaCy/BERT pipeline). The Python backend acts as a dedicated AI microservice, while Firebase serves as the primary data and auth layer for the Android app.

## SECTION 3: Android App Architecture (MVVM)

### Package Structure

The Android app follows the MVVM (Model-View-ViewModel) architecture with a clean package structure separating concerns into UI, data, and domain layers. The app uses Kotlin, Jetpack Compose for declarative UI, Navigation Compose for routing, and Retrofit for network calls.

| Package                     | Description                                              |
|-----------------------------|----------------------------------------------------------|
| com.spb.app.ui.auth         | Authentication screens (Login, Register, ForgotPassword) |
| com.spb.app.ui.job          | Job listing, details, and posting screens                |
| com.spb.app.ui.cv           | CV upload, parsing results, and management screens       |
| com.spb.app.ui.interview    | Interview scheduling, Q&A, and scoring screens           |
| com.spb.app.ui.company      | Company profile, dashboard, and management screens       |
| com.spb.app.ui.student      | Student profile, dashboard, and settings screens         |
| com.spb.app.ui.application  | Application tracking and status screens                  |
| com.spb.app.ui.notification | Notification list and preferences screens                |
| com.spb.app.ui.dashboard    | Main dashboard and analytics screens                     |
| com.spb.app.viewmodel       | All ViewModels with StateFlow state management           |
| com.spb.app.repository      | Data repositories (Auth, Job, App, Notification, AI)     |
| com.spb.app.network         | Retrofit API service interface and interceptors          |
| com.spb.app.data.model      | Data classes matching Firestore document structures      |
| com.spb.app.data.firebase   | Firebase SDK wrappers for Auth, Firestore, Storage       |
| com.spb.app.ai              | AI service layer for Gemini integration via backend      |
| com.spb.app.util            | Utility classes, extensions, constants                   |
| com.spb.app.navigation      | Navigation graph and route definitions                   |

### 9 ViewModels

| ViewModel            | State Class        | Key Functions                                 |
|----------------------|--------------------|-----------------------------------------------|
| AuthViewModel        | AuthUiState        | login, register, forgotPassword, logout       |
| JobViewModel         | JobUiState         | fetchJobs, postJob, getJobDetails, searchJobs |
| CompanyViewModel     | CompanyUiState     | getProfile, updateProfile, getDashboard       |
| ApplicationViewModel | ApplicationUiState | apply, getApplications, updateStatus          |
| DashboardViewModel   | DashboardUiState   | getStats, getRecentActivity                   |
| CVViewModel          | CVUiState          | uploadCV, parseCV, getParsingResults          |
| InterviewViewModel   | InterviewUiState   | schedule, submitAnswer, getScore              |

|                       |                     |                                                |
|-----------------------|---------------------|------------------------------------------------|
| NotificationViewModel | NotificationUiState | fetchNotifs, markRead, getUnreadCount          |
| SkillViewModel        | SkillUiState        | matchSkills, analyzeSkills, getRecommendations |

## 24 Navigation Screens

| Route                       | Screen Composable       | Description            |
|-----------------------------|-------------------------|------------------------|
| /splash                     | SplashScreen            | App entry / loading    |
| /auth/login                 | LoginScreen             | User login             |
| /auth/register              | RegisterScreen          | New user registration  |
| /auth/forgot-password       | ForgotPasswordScreen    | Password reset         |
| /student/dashboard          | StudentDashboardScreen  | Student main dashboard |
| /student/jobs               | JobListScreen           | Browse available jobs  |
| /student/job/{id}           | JobDetailScreen         | Job details & apply    |
| /student/cv                 | CVScreen                | CV upload & parsing    |
| /student/cv/result          | CVResultScreen          | Parsed CV details      |
| /student/interview          | InterviewListScreen     | Upcoming interviews    |
| /student/interview/{id}     | InterviewRoomScreen     | Live Q&A session       |
| /student/applications       | ApplicationListScreen   | Application statuses   |
| /student/recommendations    | RecommendationsScreen   | Job recommendations    |
| /student/skills             | SkillAnalysisScreen     | Skill gap analysis     |
| /student/profile            | StudentProfileScreen    | Profile management     |
| /company/dashboard          | CompanyDashboardScreen  | Company main dashboard |
| /company/jobs               | CompanyJobListScreen    | Manage posted jobs     |
| /company/jobs/post          | PostJobScreen           | Post new job           |
| /company/job/{id}           | CompanyJobDetailScreen  | View applicants        |
| /company/applicant/{id}     | ApplicantDetailScreen   | Candidate details      |
| /company/interview/schedule | ScheduleInterviewScreen | Schedule interview     |
| /company/analytics          | CompanyAnalyticsScreen  | Hiring analytics       |
| /company/profile            | CompanyProfileScreen    | Company profile        |
| /notifications              | NotificationListScreen  | Notification center    |

Network Layer - Retrofit API Endpoints (17 Endpoints)

The Android app communicates with the Python backend via Retrofit over HTTP. All AI-heavy operations are routed through the backend API to keep the Gemini API key server-side.

| Method | Route                                | Description                      |
|--------|--------------------------------------|----------------------------------|
| POST   | /api/cv/parse                        | Parse CV text via NLP            |
| POST   | /api/cv/parse-file                   | Parse uploaded CV file           |
| POST   | /api/cv/match                        | Match CV to job description      |
| POST   | /api/interview/score                 | Score interview answers          |
| POST   | /api/interview/questions             | Generate interview questions     |
| POST   | /api/interview/sentiment             | Sentiment analysis on answers    |
| POST   | /api/interview/predict               | Predict hiring probability       |
| POST   | /api/skill/match                     | Match skills to job requirements |
| POST   | /api/skill/batch-match               | Batch match multiple students    |
| POST   | /api/skill/analyze                   | Full skill analysis              |
| POST   | /api/skill/mcq/generate              | Generate MCQs for job            |
| POST   | /api/skill/mcq/parse                 | Parse MCQ text to structured     |
| POST   | /api/recommendations/jobs            | Get job recommendations          |
| POST   | /api/recommendations/learning        | Get learning recommendations     |
| POST   | /api/recommendations/resume-analysis | Resume quality analysis          |
| POST   | /api/cover-letter/generate           | Generate cover letter            |
| GET    | /health                              | Backend health check             |

# SECTION 4: Python Backend Architecture

## Complete File Structure

```
python_backend/
|-- main.py                # FastAPI app entry point
|-- requirements.txt       # Python dependency manifest
|-- .env                  # Environment variables (API keys)
|-- start_server.bat      # Windows batch launcher
|-- start_server.ps1      # PowerShell launcher
|
|-- core/
|   |-- config.py         # Server config (host, port, CORS)
|   |-- gemini.py         # GeminiClient wrapper class
|
|-- api/
|   |-- cv.py             # CV parsing & matching routes
|   |-- interview.py      # Interview scoring & questions
|   |-- skill_match.py    # Skill analysis & MCQ generation
|   |-- recommendations.py # Job & learning recommendations
|   |-- cover_letter.py   # Cover letter generation
|
|-- spb_nlp/
|   |-- __init__.py       # Module version & exports
|   |-- cv_parser.py      # CVParser (regex) + SpacyNerParser
|   |-- semantic_matcher.py # BERT SemanticMatcher + n-gram fallback
|   |-- shortlister.py    # Threshold filtering & ranking
|   |-- recommender.py    # JobRecommender hybrid scoring
|   |-- models.py         # Pydantic models (CVParseResult, Job)
|   |-- utils.py          # extract_skills, gpa, edu helpers
|   |-- evaluate.py       # Evaluation & benchmarking
|   |-- demo.py           # CLI demo pipeline
|
|-- data/
|   |-- uploads/          # Uploaded CV files
|
|-- tests/
|   |-- test_cv_parser.py # CVParser unit tests
|   |-- test_ner_parser.py # SpacyNerParser tests
|   |-- test_matcher.py   # SemanticMatcher tests
|   |-- test_shortlister.py # Shortlister tests
|   |-- test_recommender.py # Recommender tests
```

## 18 API Routes

| Method | Route                | Description                       | AI Dep   |
|--------|----------------------|-----------------------------------|----------|
| GET    | /                    | Root info / endpoint listing      | No       |
| GET    | /health              | Server health + Gemini/NLP status | No       |
| POST   | /api/cv/parse        | Parse CV text (NLP)               | No (NLP) |
| POST   | /api/cv/parse-file   | Parse uploaded CV file            | No (NLP) |
| POST   | /api/cv/match        | Match CV to job (Gemini+NLP)      | Optional |
| POST   | /api/interview/score | Score interview Q&A               | Yes      |

|      |                                      |                                 |          |
|------|--------------------------------------|---------------------------------|----------|
| POST | /api/interview/questions             | Generate questions              | Yes      |
| POST | /api/interview/sentiment             | Sentiment analysis              | Yes      |
| POST | /api/interview/predict               | Hiring prediction               | Yes      |
| POST | /api/skill/match                     | Match skills (Gemini+NLP+basic) | Optional |
| POST | /api/skill/batch-match               | Batch match students            | Optional |
| POST | /api/skill/analyze                   | Full skill analysis             | Yes      |
| POST | /api/skill/mcq/generate              | Generate MCQs                   | Yes      |
| POST | /api/skill/mcq/parse                 | Parse MCQs from text            | Yes      |
| POST | /api/recommendations/jobs            | Job recommendations             | Yes      |
| POST | /api/recommendations/learning        | Learning recommendations        | Yes      |
| POST | /api/recommendations/resume-analysis | Resume analysis                 | Yes      |
| POST | /api/cover-letter/generate           | Generate cover letter           | Yes      |

## Core Configuration (core/config.py)

Configuration is loaded from a .env file via python-dotenv. Key settings include GEMINI\_API\_KEY for Gemini AI access, GEMINI\_MODEL (default: gemini-flash-lite-latest), SERVER\_HOST and SERVER\_PORT for the uvicorn server, and DEFAULT\_MATCH\_THRESHOLD (50.0). CORS is configured to allow all origins for development flexibility.

## Gemini Integration Pattern (core/gemini.py)

The GeminiClient class wraps the google-genai SDK with three generation methods: generate() for plain text output, generate\_json() for structured JSON responses, and generate\_with\_images() for multimodal inputs. The client is lazy-initialized and checks API key availability via is\_available(). All AI API routes check this flag and return HTTP 503 if Gemini is not configured. JSON responses are cleaned of markdown fences before parsing to handle varied model outputs.

## SECTION 5: Database Schema (Firestore)

Firestore is used as the primary NoSQL database with 6 collections. All collections use auto-generated document IDs generated by Firestore. The schema is designed for denormalized reads to minimize joins, with some data duplication for performance.

### 1. students (18 fields)

| Field                   | Type          | Description                              |
|-------------------------|---------------|------------------------------------------|
| studentId               | string        | Unique student identifier (Firebase UID) |
| name                    | string        | Full name of the student                 |
| email                   | string        | University email address                 |
| phone                   | string        | Contact phone number                     |
| password                | string        | Hashed password (Firebase Auth managed)  |
| major                   | string        | Academic major/department                |
| cgpa                    | number        | Cumulative GPA (0.0 - 4.0)               |
| skills                  | array<string> | List of skills (parsed from CV)          |
| experience              | string        | Work experience summary text             |
| education               | string        | Education history text                   |
| certifications          | array<string> | Professional certifications              |
| projects                | array<string> | Academic/personal projects               |
| resumeUrl               | string        | Firebase Storage URL for CV file         |
| profileImageUrl         | string        | Profile photo URL                        |
| fcmToken                | string        | Firebase Cloud Messaging token           |
| createdAt               | timestamp     | Account creation timestamp               |
| updatedAt               | timestamp     | Last profile update timestamp            |
| notificationPreferences | map           | Push notification preferences            |



## 2. companies (14 fields + nested ContactPerson)

| Field         | Type      | Description                              |
|---------------|-----------|------------------------------------------|
| companyId     | string    | Unique company identifier (Firebase UID) |
| companyName   | string    | Registered company name                  |
| email         | string    | Company contact email                    |
| phone         | string    | Company phone number                     |
| password      | string    | Hashed password (Firebase Auth managed)  |
| industry      | string    | Industry sector (e.g. IT, Finance)       |
| description   | string    | Company description / about us           |
| website       | string    | Company website URL                      |
| location      | string    | Office/campus location                   |
| logoUrl       | string    | Company logo image URL                   |
| fcmToken      | string    | FCM push notification token              |
| createdAt     | timestamp | Account creation timestamp               |
| updatedAt     | timestamp | Last profile update                      |
| isVerified    | boolean   | Verification status                      |
| contactPerson | map       | Nested object:                           |
| .name         | string    | Contact person name                      |
| .designation  | string    | Contact person job title                 |
| .phone        | string    | Contact person phone                     |
| .email        | string    | Contact person email                     |

## 3. jobs (22+ fields, nested MCQ)

| Field              | Type          | Description                         |
|--------------------|---------------|-------------------------------------|
| jobId              | string        | Unique job identifier (auto-ID)     |
| companyId          | string        | Reference to companies collection   |
| companyName        | string        | Company display name (denormalized) |
| title              | string        | Job title / position name           |
| description        | string        | Detailed job description            |
| requiredSkills     | array<string> | Required skill keywords             |
| preferredSkills    | array<string> | Nice-to-have skills                 |
| requiredEducation  | string        | Minimum education requirement       |
| minimumGpa         | number        | Minimum GPA threshold               |
| experienceRequired | string        | Years/level of experience           |

|                     |               |                                    |
|---------------------|---------------|------------------------------------|
| jobType             | string        | Full-time / Part-time / Internship |
| workMode            | string        | On-site / Remote / Hybrid          |
| location            | string        | Job location                       |
| salaryRange         | string        | Salary range string                |
| vacancies           | number        | Number of open positions           |
| applicationDeadline | timestamp     | Last date to apply                 |
| status              | string        | Active / Closed / Draft            |
| mcq                 | map           | Nested MCQ object:                 |
| .question           | string        | MCQ question text                  |
| .options            | array<string> | MCQ answer choices                 |
| .correctAnswerIndex | number        | Index of correct answer            |
| .answer             | string        | Correct answer text                |
| createdAt           | timestamp     | Job posting timestamp              |
| updatedAt           | timestamp     | Last update timestamp              |

#### 4. applications (28+ fields, nested InterviewData & ActivityLog)

| Field           | Type          | Description                                 |
|-----------------|---------------|---------------------------------------------|
| applicationId   | string        | Unique application ID (auto-ID)             |
| studentId       | string        | Reference to students collection            |
| studentName     | string        | Applicant name (denormalized)               |
| jobId           | string        | Reference to jobs collection                |
| jobTitle        | string        | Job title applied for (denormalized)        |
| companyId       | string        | Reference to companies collection           |
| companyName     | string        | Company name (denormalized)                 |
| status          | string        | Pending / Shortlisted / Accepted / Rejected |
| matchScore      | number        | Semantic match score (0-100)                |
| coverLetter     | string        | Applicant's cover letter text               |
| appliedAt       | timestamp     | Application submission timestamp            |
| updatedAt       | timestamp     | Status update timestamp                     |
| notes           | string        | Company's internal notes                    |
| interviewData   | map           | Nested interview object:                    |
| .scheduledAt    | timestamp     | Interview schedule time                     |
| .interviewType  | string        | Technical / HR / Behavioral                 |
| .status         | string        | Scheduled / Completed / Cancelled           |
| .questions      | array<string> | Interview questions asked                   |
| .answers        | array<string> | Candidate's answers                         |
| .score          | number        | Interview score (0-100)                     |
| .feedback       | string        | Interviewer feedback text                   |
| .meetLink       | string        | Google Meet / Zoom link                     |
| .sentimentScore | number        | Sentiment analysis score                    |
| .isAIGenerated  | boolean       | Was AI used for scoring?                    |
| activityLog     | array<map>    | Array of activity entries:                  |
| [].action       | string        | Action performed                            |
| [].timestamp    | timestamp     | When action occurred                        |
| [].details      | string        | Action details/notes                        |

#### 5. notifications (15 fields)

| Field          | Type   | Description            |
|----------------|--------|------------------------|
| notificationId | string | Unique notification ID |

|                      |           |                                         |
|----------------------|-----------|-----------------------------------------|
| userId               | string    | Target user (student or company)        |
| userRole             | string    | student / company                       |
| type                 | string    | application_update / interview / system |
| title                | string    | Notification title                      |
| message              | string    | Notification body text                  |
| data                 | map       | Additional payload data                 |
| relatedJobId         | string    | Optional job reference                  |
| relatedApplicationId | string    | Optional application reference          |
| isRead               | boolean   | Read/unread status                      |
| createdAt            | timestamp | Notification creation time              |
| readAt               | timestamp | When user read the notification         |
| priority             | string    | high / normal / low                     |
| imageUrl             | string    | Optional image URL                      |
| deepLink             | string    | App navigation deep link                |

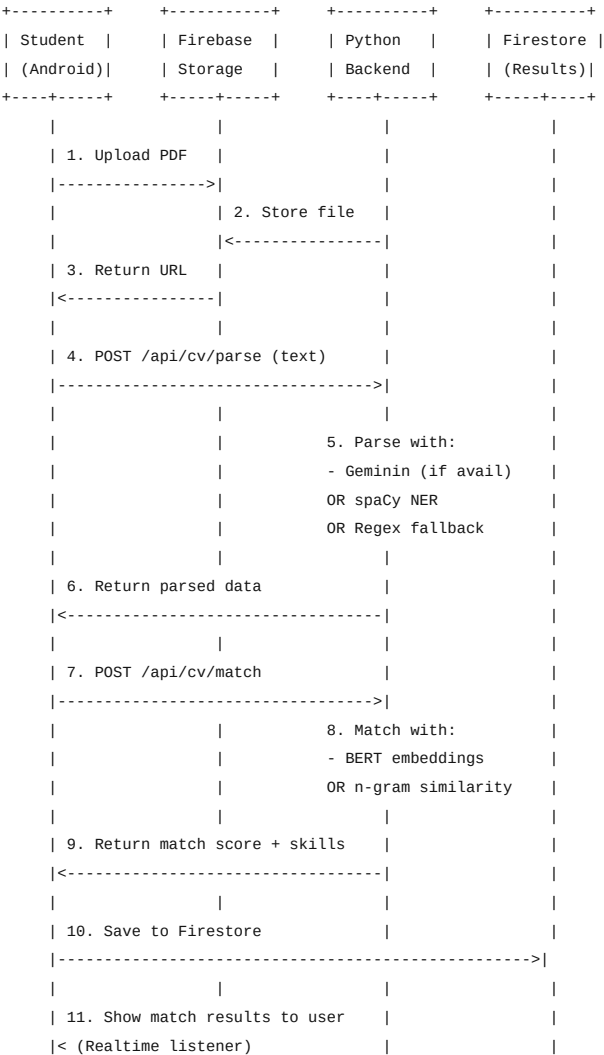
## 6. pending\_notifications (3 fields)

| Field          | Type   | Description                    |
|----------------|--------|--------------------------------|
| notificationId | string | Unique notification ID         |
| userId         | string | Target user ID                 |
| data           | map    | Full notification data payload |

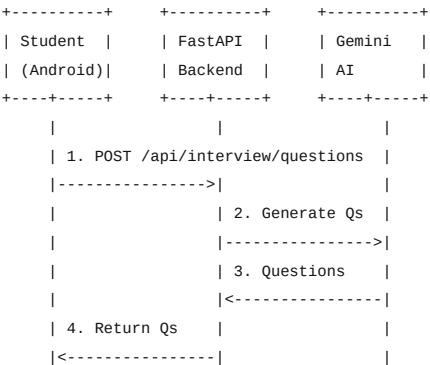
Relationships: students.applications -> jobs (via studentId/jobId). companies.jobs -> companies (via companyId). applications.jobId -> jobs.jobId. applications.studentId -> students.studentId. notifications.userId -> students.studentId OR companies.companyId. pending\_notifications is a staging collection for FCM delivery tracking.

## SECTION 6: Data Flow Diagrams

### Flow 1: CV Upload -> Parse -> Match

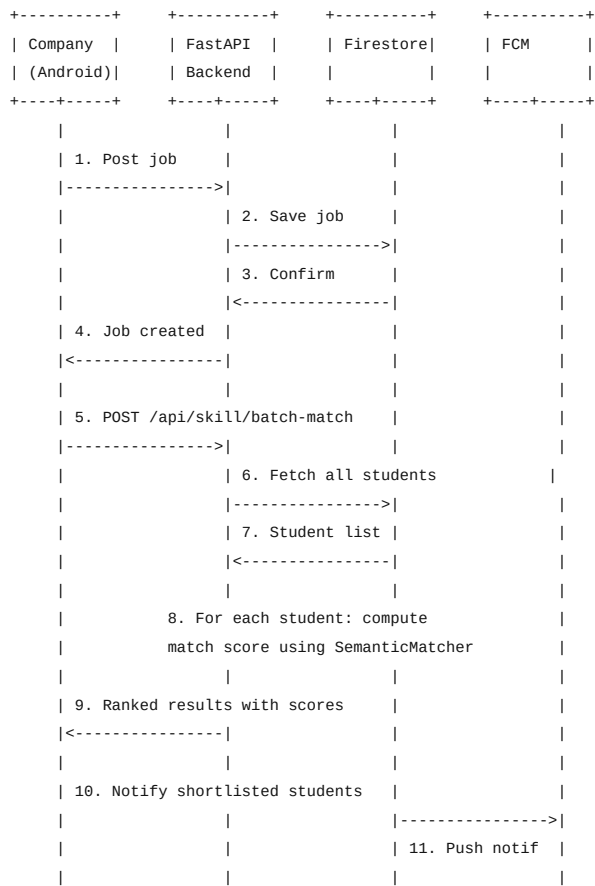


### Flow 2: Interview Flow (Question -> Answer -> Score)

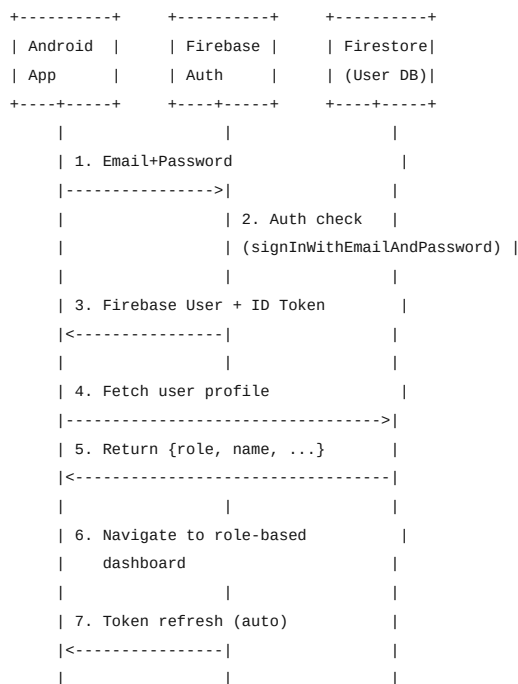


```
|           |           |
| 5. Student answers (text) |
|           |           |
| 6. POST /api/interview/score |
|----->|           |
|           | 7. Evaluate |
|           |----->|
|           | 8. Score+feedback|
|           |<-----|
| 9. Return score, strengths, |
|   improvements, tone, clarity |
|<-----|           |
|           |           |
| 10. POST /api/interview/predict |
|----->|           |
|           | 11. Predict |
|           |----->|
|           | 12. Probability |
|           |<-----|
| 13. Show hiring prediction |
|<-----|           |
```

### Flow 3: Job Posting -> Batch Matching -> Notification



### Flow 4: Authentication Flow



|                                |  |
|--------------------------------|--|
| 8. Token attached to API calls |  |
| via Retrofit interceptor       |  |
|                                |  |
| For registration:              |  |
| 1. FirebaseAuth.createUser()   |  |
| 2. Firestore document created  |  |
| in students or companies       |  |
| collection                     |  |



## SECTION 7: AI Architecture & Fallback Chain

The SPB system implements a multi-tier AI architecture designed for resilience. The primary AI provider is Google Gemini API, with automatic fallback through spaCy NER, BERT embeddings, and finally regex-based parsers. This ensures the system continues to function even when external API services are unavailable.

### Tier 1: Gemini API (google-genai SDK)

Model: gemini-flash-lite-latest (configurable via GEMINI\_MODEL env var). Used for CV parsing (structured JSON extraction), interview scoring and feedback, skill analysis with career recommendations, MCQ generation and parsing, job recommendations with ranking, sentiment analysis, and cover letter generation. Requires GEMINI\_API\_KEY in .env file. Returns HTTP 503 if unavailable.

### Tier 2: spaCy NER (SpacyNerParser)

Model: en\_core\_web\_lg. Used as fallback for CV parsing when Gemini is unavailable or for offline operation. Extracts entities: PERSON (candidate name), ORG (organizations), PRODUCT (skills/tools). Also extracts structured fields including email (regex), phone (regex), skills (NER + keyword list), education, experience, GPA, certifications, and projects. Falls back to regex-based CVParser if spaCy model not loaded.

### Tier 3: Regex-based Parser (CVParser)

Zero-dependency CV parser that uses regex patterns and a dictionary of 172 skill keywords. Extracts candidate name (line-based heuristics), email, phone, GPA, education, experience, projects, and certifications. Always available regardless of external dependencies.

### Semantic Matching Pipeline

Primary: sentence-transformers (all-MiniLM-L6-v2) producing 384-dimensional embeddings. Cosine similarity is computed between CV and job description embeddings. The match score combines: skill keyword overlap (0-60%), semantic similarity (0-10%), experience score (0-15%), education score (0-10%), and extra bonus (0-5%) for a total of 0-100.

### Fallback: Character n-gram Similarity

When BERT embeddings cannot be computed (e.g., model not downloaded), the system falls back to character n-gram cosine similarity (bigrams + trigrams). This is a zero-dependency approach using pure Python collections.Counter and math operations. Used by the SemanticMatcher class.

### AI Tier by Endpoint

| Endpoint                      | AI Tier (Primary -> Fallback)       |
|-------------------------------|-------------------------------------|
| POST /api/cv/parse            | NLP (SpacyNerParser -> CVParser)    |
| POST /api/cv/parse-file       | NLP (SpacyNerParser -> CVParser)    |
| POST /api/cv/match            | NLP SemanticMatcher -> Basic n-gram |
| POST /api/interview/score     | Gemini (required)                   |
| POST /api/interview/questions | Gemini (required)                   |
| POST /api/interview/sentiment | Gemini (required)                   |
| POST /api/interview/predict   | Gemini (required)                   |

|                                              |                                    |
|----------------------------------------------|------------------------------------|
| POST /api/skill/match                        | SemanticMatcher -> Gemini -> Basic |
| POST /api/skill/batch-match                  | SemanticMatcher -> Basic overlap   |
| POST /api/skill/analyze                      | Gemini (required)                  |
| POST /api/skill/mcq/generate                 | Gemini (required)                  |
| POST /api/skill/mcq/parse                    | Gemini (required)                  |
| POST /api/recommendations/jobs               | Gemini (required)                  |
| POST /api/recommendations/learning           | Gemini (required)                  |
| POST<br>/api/recommendations/resume-analysis | Gemini (required)                  |
| POST /api/cover-letter/generate              | Gemini (required)                  |

## SECTION 8: Security Architecture

### API Key Management

The Gemini API key is stored exclusively in the .env file on the backend server. It is NEVER embedded in the Android app code or APK. The Android app communicates with Gemini indirectly through the Python FastAPI backend, ensuring the API key remains server-side only.

### Firebase Security Rules

```
// students collection
match /students/{userId} {
  allow read, write: if request.auth != null
    && request.auth.uid == userId;
}

// companies collection
match /companies/{companyId} {
  allow read, write: if request.auth != null
    && request.auth.uid == companyId;
}

// jobs collection - public read, company write
match /jobs/{jobId} {
  allow read: if request.auth != null;
  allow create: if request.auth != null
    && request.auth.token.role == "company";
  allow update, delete: if request.auth != null
    && resource.data.companyId == request.auth.uid;
}

// applications - student sees own, company sees own jobs
match /applications/{appId} {
  allow read: if request.auth != null
    && (request.auth.uid == resource.data.studentId
    || request.auth.uid == resource.data.companyId);
  allow create: if request.auth != null;
  allow update: if request.auth != null
    && request.auth.uid == resource.data.companyId;
}

// notifications - user sees own
match /notifications/{notifId} {
  allow read, write: if request.auth != null
    && request.auth.uid == resource.data.userId;
}
```

### Network Security Config (Android)

The Android app uses a network security config that permits cleartext HTTP traffic to local network IPs (192.168.x.x) for development purposes only. In production, all traffic is HTTPS. The backend CORS is configured to allow all origins for development, with production deployment restricting to the Android app's origin.

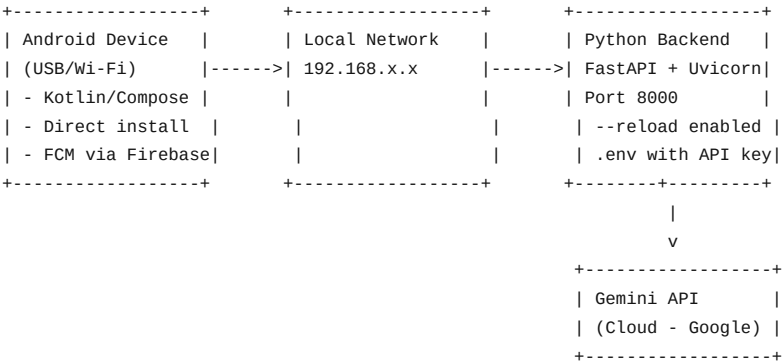
### Additional Security Measures

1. No direct Gemini calls from Android - API key stays server-side.
2. Firebase Auth manages all user authentication with email/password.
3. Firestore security rules enforce document-level access control.

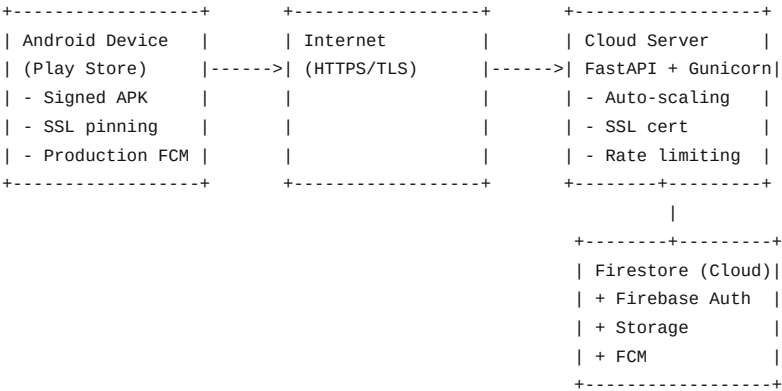
4. Retrofit interceptor attaches Firebase ID token to every API request.
5. Backend validates and sanitizes all user inputs before processing.
6. File uploads restricted to PDF format for CV files.
7. Environment variables (.env) excluded from version control.
8. CORS middleware allows configurable origins.

## SECTION 9: Deployment Architecture

### Development Environment



### Production Environment



### Server Startup (Development)

The Python backend is started using uvicorn with the --reload flag for hot reload during development. The server binds to the configured SERVER\_HOST (default: 192.168.0.114) and SERVER\_PORT (default: 8000). The CORS middleware allows all origins for development. Startup scripts are provided as start\_server.bat and start\_server.ps1 for Windows.

```
# From python_backend/ directory:
uvicorn main:app --host 192.168.0.114 --port 8000 --reload

# Or using the startup scripts:
.\start_server.bat    # Windows batch
.\start_server.ps1   # PowerShell
```

### Environment Configuration (.env)

```
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_MODEL=gemini-flash-lite-latest
SERVER_HOST=192.168.0.114
```

SERVER\_PORT=8000